

An Adaptive Post-Quantum Cryptography Architecture for IoT: Deployment and Evaluation for Quantum-Safe Networks

Dong-Quan Nguyen, Nhat-Phat Huynh, and Quan Le

Abstract—This paper proposes a capability-tiered architecture for deploying post-quantum cryptography (PQC) in resource-heterogeneous Internet-of-Things (IoT) systems. Instead of requiring every IoT device to execute all PQC primitives locally, the proposed architecture enforces cryptographic functional partitioning based on device capabilities, network roles, and trust assumptions. In this model, resource-constrained edge nodes maintain lightweight local-domain security and delegate intensive PQC operations to a managed gateway node. Conversely, high-capability edge nodes and gateway nodes natively execute ML-KEM, ML-DSA, and SLH-DSA when platform-specific profiling confirms acceptable latency, memory, and energy costs. The evaluation relies on empirical hardware measurements on a Lichee RV Nano W (RV64) platform across various scenarios, ranging from standalone benchmarks to concurrent, batched node-to-server communication sessions. For digital signature algorithms, the dominant transmitted object is the signature itself. For ML-KEM, transmitted data is structured under the KEM/DEM framework, combining a KEM ciphertext with an authenticated-encryption payload. The empirical results indicate that ML-KEM and ML-DSA are optimized for frequent operations such as session establishment and authentication. Conversely, SLH-DSA is reserved for selective assumption-diverse operations, including firmware validation, secure boot, and Root-of-Trust verification. Thus, this multi-tiered deployment paradigm balances post-quantum security, latency, bandwidth, and system-level adaptability across IoT networks.

Index Terms—Internet of Things, post-quantum cryptography, ML-KEM, ML-DSA, SLH-DSA

I. INTRODUCTION

The rapid expansion of the Internet of Things (IoT) has created large-scale networks in which data are continuously exchanged among edge nodes, gateway nodes, and cloud services. These systems are increasingly deployed in smart homes, industrial monitoring, personal healthcare, intelligent transportation, and digital infrastructure. Because IoT data may represent environmental conditions, device state, user identity, or physical control commands, confidentiality, integrity, and authentication are fundamental security requirements rather than optional features.

IoT security is challenging due to the wide disparity in hardware capabilities among deployed devices. A realistic IoT network typically encompasses resource-constrained edge nodes, high-capability edge nodes, gateway nodes, and cloud services. While high-capability edge nodes, gateway nodes, and

cloud services can execute relatively expensive cryptographic protocols, resource-constrained edge nodes are restricted by small SRAM, limited flash memory, low CPU frequency, battery-powered operation, and low-bandwidth wireless communication. Therefore, applying a uniform public-key cryptographic mechanism across every device is usually impractical.

The prospect of cryptographically relevant quantum computers creates a migration challenge for this heterogeneous ecosystem. RSA and elliptic-curve cryptography rely on problems that Shor's algorithm would solve in polynomial time on such a computer [1]. Long-lived sensitive data may therefore be exposed to harvest-now-decrypt-later attacks, while the energy cost of replacement cryptography remains a separate deployment constraint on battery-powered nodes [2]. Prior work has examined lattice-based cryptography for edge devices [3], [4] and energy profiling of PQC implementations [5]. NIST has standardized the Module-Lattice-Based Key-Encapsulation Mechanism (ML-KEM) in FIPS 203 [6], the Module-Lattice-Based Digital Signature Algorithm (ML-DSA) in FIPS 204 [7], and the Stateless Hash-Based Digital Signature Algorithm (SLH-DSA) in FIPS 205 [8].

Direct deployment of these primitives on IoT devices presents significant challenges because performance, memory, communication, and energy costs depend on both the implementation and the target platform [9]. Selected small-signature SLH-DSA parameter sets trade smaller signatures for higher signing cost, motivating implementation-specific evaluation on low-power devices [10]. Empirical evaluations likewise show substantial variation across heterogeneous platforms [11] and constrained-device setups [12]. Software work has reduced the memory and processing cost of lattice-based schemes on microcontrollers [13], [14], and pqm4 provides a Cortex-M4 testing and benchmarking framework [15]. Hardware research includes lattice-oriented processors for IoT workloads [16], accelerators for ML-KEM and ML-DSA [17], and system-level integration on OpenTitan [18].

Figure 1 summarizes the gateway-assisted and native-PQC deployment paths.

This paper therefore proposes a capability-tiered IoT architecture that distributes cryptographic responsibilities among edge nodes, gateway nodes, and cloud services. Resource-constrained edge nodes use authenticated symmetric protection for local communication, while a managed gateway node can terminate the local trust domain and establish a cloud-facing PQC channel. High-capability edge nodes may execute ML-KEM and ML-DSA locally after platform-specific profiling. This

Dong-Quan Nguyen, Nhat-Phat Huynh, and Quan Le are with the University of Information Technology, Vietnam National University Ho Chi Minh City, Ho Chi Minh City, Vietnam (e-mail: {24521438, 24521294, 24521429}@gm.uit.edu.vn).

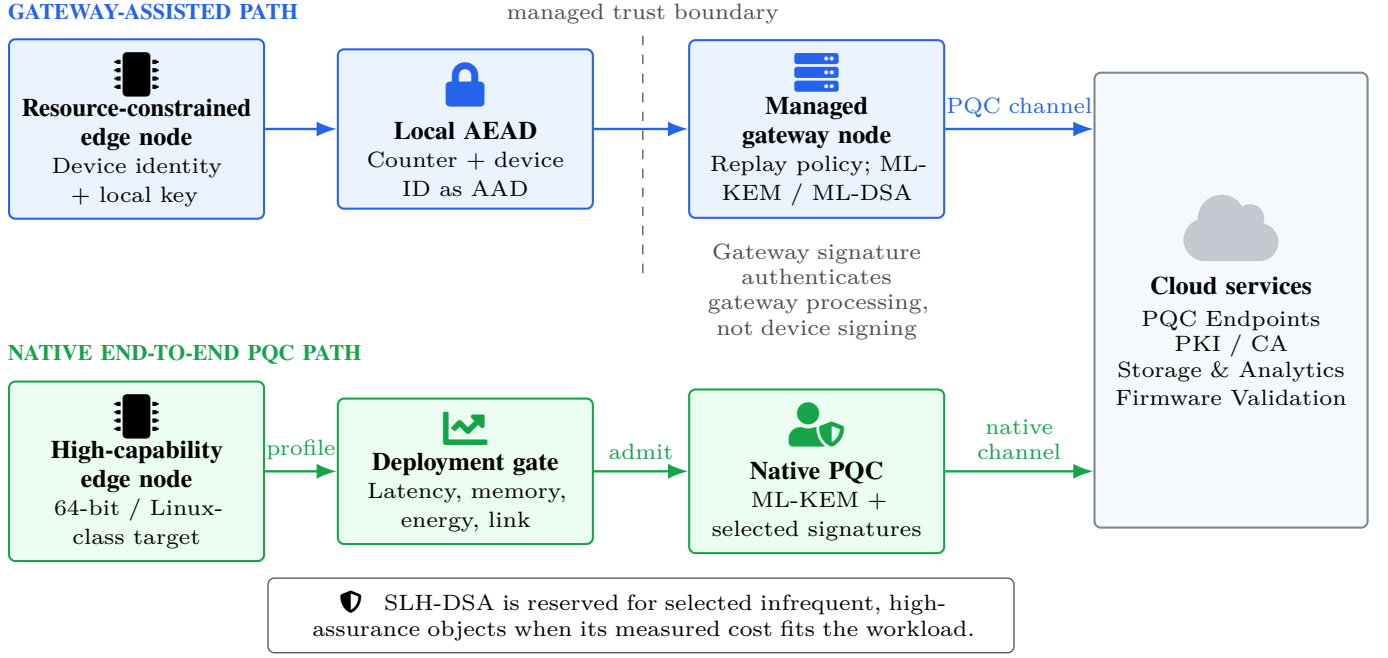


Fig. 1. Adaptive capability-tiered PQC architecture. Resource-constrained edge nodes use an authenticated local path through a managed gateway node, whereas high-capability edge nodes may use native end-to-end PQC after platform-specific profiling. The tiered design denotes compositional placement and gateway provisioning; throughput scaling is not claimed as measured.

structure retains compatibility with heterogeneous devices while making the gateway trust boundary explicit. This compositional architecture is adaptive: additional nodes can be assigned to a native-PQC path after profiling or to provisioned gateway nodes without imposing one public-key workload on every device. Gateway capacity, queueing, and multi-board performance remain evaluation targets.

To evaluate the proposed architecture, the paper utilizes a physical edge-device stress-test scenario on a Lichee RV Nano W RV64 board. ML-KEM, ML-DSA, and six SLH-DSA small-signature parameter sets are evaluated under repeated batched concurrent workloads. Digital signatures constitute the primary cryptographic object for ML-DSA and SLH-DSA, whereas ML-KEM is analyzed as a KEM/DEM composition consisting of a KEM ciphertext and an authenticated-encryption payload. The experiments examine workload-specific placement rather than claiming a platform-independent ranking.

The main contributions of this paper are summarized as follows. First, it proposes a capability-tiered architecture that organizes PQC placement across edge nodes, gateway nodes, and cloud services according to computational capability, network role, and trust assumptions. Second, it formalizes conditional deployment rules that keep resource-constrained edge-node protection distinct from gateway-authenticated processing and native end-to-end PQC. Third, it reports latency, communication-object, and stress-test records for research-fork implementations mapped to the ML-KEM, ML-DSA, and SLH-DSA families, while separating native measurements from QEMU portability checks. Finally, it derives workload-specific placement recommendations and states the evidence required before those recommendations can be transferred to another IoT system.

II. BACKGROUND AND RELATED WORK

A. Standardized Post-Quantum Cryptography

PQC is a central research direction in applied cryptography because sufficiently capable quantum computers would undermine the mathematical foundations of RSA and elliptic-curve cryptography [3], [4]. This work evaluates ML-KEM for key encapsulation, ML-DSA for lattice-based digital signatures, and SLH-DSA for stateless hash-based digital signatures. NIST specifies these algorithms in FIPS 203, FIPS 204, and FIPS 205, respectively [6], [7], [8]. ML-KEM establishes a shared secret, while ML-DSA and SLH-DSA provide signatures under different security constructions. Their parameter sets differ in key, ciphertext or signature size, computational cost, and suitability for a given device and workload.

ML-KEM, derived from CRYSTALS-Kyber, is a key encapsulation mechanism designed to establish a shared secret over an untrusted channel. Unlike conventional encryption algorithms, ML-KEM does not directly encrypt application data. Instead, it produces a shared secret together with a KEM ciphertext. The shared secret can then be used by a Data Encapsulation Mechanism (DEM)—typically a symmetric authenticated-encryption scheme such as AES-GCM or ChaCha20-Poly1305—to protect the application payload. For IoT communication, the transmitted data therefore include not only the KEM ciphertext but also the encrypted payload and the associated overhead of the symmetric encryption scheme.

ML-DSA, derived from CRYSTALS-Dilithium, is a lattice-based digital signature algorithm used to provide authentication and data integrity. Under the implementations and workloads evaluated in this paper, it has smaller signatures and lower reported signing cost than the selected SLH-DSA small-

signature parameter sets. The proposed architecture therefore considers ML-DSA for recurring operations such as device authentication, command verification, and integrity protection for regularly exchanged data.

SLH-DSA, derived from SPHINCS+, is a stateless hash-based alternative to lattice-based signatures. The selected small-signature parameter sets produce larger signatures and have higher reported signing cost than the evaluated ML-DSA configurations. The architecture therefore considers them for infrequent operations such as firmware validation, secure boot, update authorization, and root-of-trust establishment, subject to system-specific assurance requirements. Table I summarizes these proposed roles and trade-offs.

B. PQC on Resource-Constrained Edge Nodes

Previous research has examined the implementation of PQC on microcontrollers and embedded platforms, especially on devices with limited memory, ARM Cortex-M processors, and wireless IoT nodes. Multiple publications have reported implementations and optimizations of lattice-based algorithms for constrained microcontroller platforms [13], [14], [15], [16]. Other evaluations have focused on ML-KEM and NTRU in IoT-oriented settings [19], while more recent investigations on Cortex-M0+ microcontrollers, such as the RP2040 [20], [21], extend earlier Cortex-M4 evaluations of ML-KEM and ML-DSA [14], [15]. Other work has investigated post-quantum TLS 1.3, authentication latency, and session-establishment overhead in embedded communication stacks [1], [22], [23], [24], [25]. Analyses of energy and computational cost further show that the practical use of PQC on battery-powered IoT devices depends not only on execution time, but also on memory use, communication overhead, and duty-cycle constraints [10], [11], [12], [24], [26]. Deployment-oriented research has also considered secure software updates and signature-based authentication on platforms such as LoRa and ESP32 [27], [28], [29]. Overall, KEMs and lattice-based signature schemes can be practical on selected constrained platforms after careful optimization. In contrast, stateless hash-based signatures remain difficult to use frequently on such devices because of their large signatures and high signing cost [8].

However, showing that an algorithm can run on a particular microcontroller does not mean that it should be deployed on every resource-constrained edge node. In real deployments, these nodes often operate with limited battery capacity, low-bandwidth wireless links, and strict latency requirements. They must also perform sensing, control, communication, and storage tasks in parallel with security processing. As a result, a native-PQC design may place too much load on the device even when an isolated benchmark completes successfully. Therefore, PQC deployment decisions should be based on system-level requirements rather than isolated algorithm runtimes.

This work takes a conservative approach to PQC deployment. Native PQC is treated as an optional feature and is enabled only after platform-specific evaluation. Although a PQC algorithm may run successfully on a device, this does not necessarily mean it is suitable for practical deployment in a complete IoT

system [28]. This approach provides a deployment guideline whose system-level performance must be evaluated in the target system.

C. PQC on Gateway Nodes and 64-bit Platforms

Gateway nodes are architecturally well-suited for computationally intensive PQC processing. Compared with battery-powered edge nodes, gateway nodes typically possess larger memory footprints, higher CPU capabilities, stable power supplies, and more reliable network connectivity. They can terminate or initiate post-quantum-protected channels, verify digital signatures, aggregate local data streams, and enforce security policies before forwarding data to cloud services.

The 64-bit platforms, such as x86_64, ARM64, and RV64, represent critical deployment targets for gateway nodes and high-capability edge nodes. Cross-platform benchmarking is motivated by the inherent architectural heterogeneity of modern IoT systems [9]. For hardware acceleration, prior literature encompasses lattice-oriented co-processors, instruction-set extensions, HW/SW co-designs, and hash-accelerator implementations for cryptographic workloads [30], [31], [32], [33]. Within the open-hardware ecosystem, RISC-V processor architectures featuring custom accelerators or vector units have been proposed for ML-KEM, ML-DSA, and SLH-DSA to mitigate latency and power bottlenecks [34], [35].

D. Research Gap

Within the literature reviewed for this study, four deployment-level and system-level gaps receive less consistent treatment than isolated algorithm benchmarks:

Limited Deployment-Oriented Guidance: Many existing works demonstrate the feasibility of individual PQC primitives, but fewer translate these results into capability-aware placement guidance across resource-constrained edge nodes, high-capability edge nodes, gateway nodes, and cloud services.

Limited System-Level Coverage and Runtime Visibility: Bare-metal microbenchmarks do not capture scheduling, process orchestration, buffering, and kernel activity on Linux-capable devices. The present stress test reports elapsed time, user and system CPU time, and maximum RSS at the system level, but does not attribute the observed values to individual runtime causes.

Limited Concurrent Physical-Node Evaluation: Most prior evaluations emphasize isolated cryptographic calls. Fewer report repeated batched concurrent workloads together with operating-system activity, message-flow handling, and communication-object costs on a physical edge node.

Asymmetric Cross-ISA Evidence: Native and emulated results are often available for different architectures and cannot be treated as directly comparable physical measurements. This paper therefore keeps native measurements separate from functional emulation and relative-trend checks.

This paper combines a capability-tiered IoT architecture and placement guidance with system-level runtime metrics, repeated batched concurrent stress testing, and explicit separation of native and emulated evidence. The deployment question is

TABLE I
ROLE COMPARISON OF STANDARDIZED PQC PRIMITIVES IN IoT SYSTEMS

Algorithm	Standard	Primitive Type	Main Advantage	Main Limitation	Recommended Role
ML-KEM	FIPS 203 [6]	Key encapsulation mechanism	Low latency and moderate ciphertext size	Requires polynomial arithmetic and careful implementation	Frequent session establishment
ML-DSA	FIPS 204 [7]	Digital signature	Efficient signing and verification for regular authentication	Larger keys and signatures than classical ECC	Device and data authentication
SLH-DSA	FIPS 205 [8]	Stateless hash-based signature	Conservative hash-based security assumption	Large signatures and high signing latency	Firmware validation, secure boot, and Root-of-Trust

which primitive and placement are compatible with the measured resource and workload constraints, not which primitive is universally optimal. Figure 2 summarizes the reviewed literature streams, these four deployment-level and system-level gaps, and the bounded responses reported in this paper.

III. CAPABILITY-TIERED IoT ARCHITECTURE

A. Threat Model and Deployment Assumptions

The proposed architecture assumes an adversary capable of observing, replaying, injecting, or modifying network communications between IoT devices, gateway nodes, and cloud services. The adversary may also store intercepted data for future decryption, motivating the adoption of post-quantum cryptography. Physical compromise of every deployed device is not assumed, although resource-constrained devices are generally more vulnerable than managed infrastructure.

The local communication domain is not considered fully trusted. Therefore, local data exchanges should still be protected using symmetric authenticated encryption. Offloading PQC

operations reduces the computational and memory burden on constrained devices but does not eliminate the need for local authentication and access control. Cloud services are assumed to have sufficient resources to support all PQC algorithms and long-term key management. These assumptions provide a practical basis for supporting legacy and constrained IoT devices without assuming that every device must execute all PQC algorithms locally.

B. Trust, Identity, and Replay-Protection Model

The gateway-assisted deployment introduces a clear trust boundary. In this model, the cloud does not receive a digital signature directly from every constrained device. Instead, the gateway authenticates devices within the local network, applies security policies, and communicates with the cloud through a post-quantum-protected channel. This model is suitable when the gateway is managed and maintained by the system operator, such as in industrial networks, building-control systems, or local data-collection hubs.

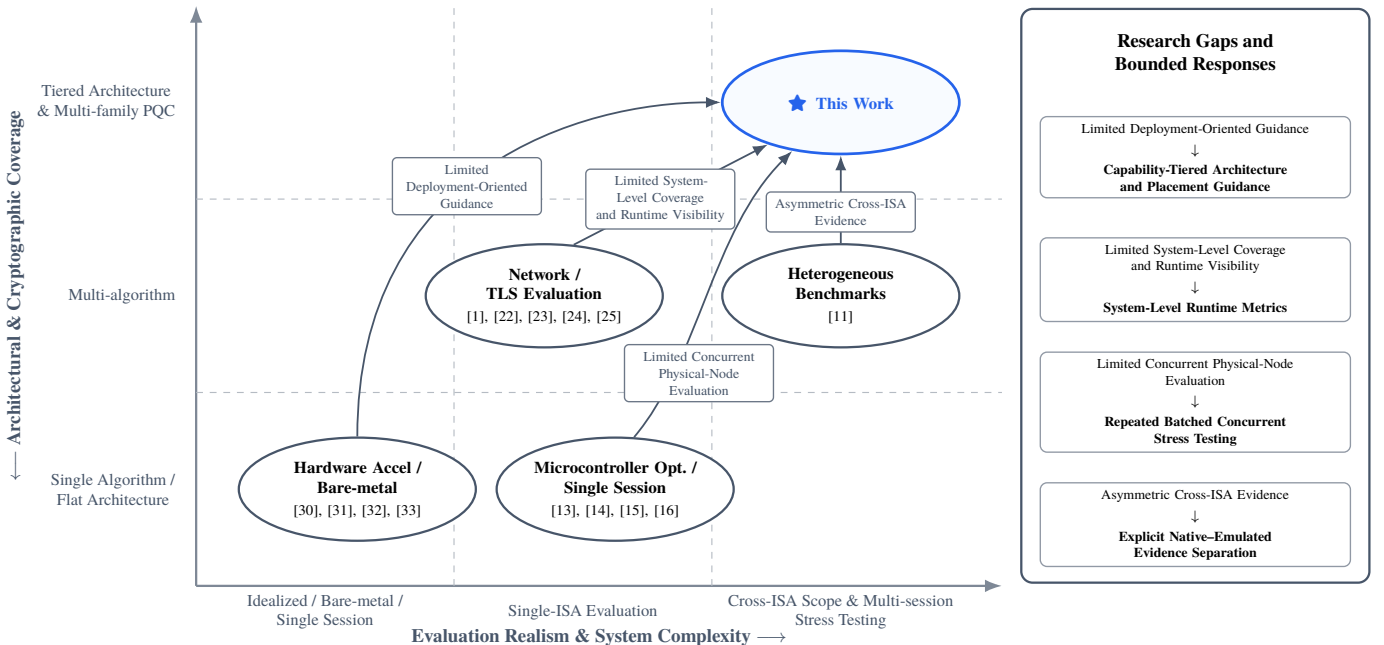


Fig. 2. Conceptual taxonomy: Research-gap map and the positioning of this work. The map categorizes reviewed literature streams along axes of evaluation realism and architectural coverage, then proposes bounded responses to four deployment-level and system-level gaps.

Each device should have its own identity within the local network. This identity may use a pre-shared symmetric key, a device-specific credential, or another credential format suitable for the target hardware. Local messages are protected using AEAD with device identity and replay-protection information included in the authenticated data. To mitigate the threat of Grover's algorithm halving the effective symmetric key space, this local channel must use a 256-bit symmetric key (e.g., AES-256-GCM or ChaCha20-Poly1305). The gateway should reject stale counters and unexpected message types to reduce replay and injection attacks. When data are forwarded to the cloud, the gateway may report the verified device identity or sign an aggregated record using its own credential. In the latter case, the signature represents authentication performed by the gateway, not a direct signature from the device.

High-capability edge nodes may run ML-KEM and ML-DSA directly. These nodes can establish post-quantum-protected sessions with cloud services and authenticate selected messages without relying on a gateway node. The architecture therefore supports two deployment paths: gateway-assisted protection for resource-constrained edge nodes and native end-to-end PQC for high-capability edge nodes.

C. System Model

The proposed system architecture encompasses three logical tiers: edge nodes, gateway nodes, and cloud services. The edge tier is divided into resource-constrained edge nodes and high-capability edge nodes. Resource-constrained edge nodes represent low-power devices with limited CPU capability, memory, energy, or network bandwidth, such as basic sensors and simple controllers. High-capability edge nodes are platforms admitted to the native-PQC path after workload-specific profiling and are typically driven by 64-bit processors running lightweight operating systems or Linux-based environments. Gateway nodes interface the local IoT domain with the Internet and function as strategic security anchors. Cloud services provide elastic storage, analytics, remote management, and high-resource PQC endpoints.

The architecture enforces an asymmetric distribution of cryptographic responsibilities rather than assigning a uniform workload to every device. Each architectural tier is allocated a specific role aligned with its resource profile. The resource-constrained edge-node class minimizes public-key cryptographic operations. The high-capability edge-node class executes selected PQC primitives natively. The gateway-node tier handles computationally intensive PQC processing and secures communication channels directed toward the cloud. Finally, the cloud-service tier supports all standardized primitives and long-term security policies. This structural partitioning across the local, edge, and cloud planes is expanded in Figure 3.

D. Design Principles

The proposed architecture is based on three design principles. First, PQC placement should match the capability and workload of each device. Resource-constrained and battery-powered

devices should not be assigned the same default public-key workload as gateway nodes or cloud services. Second, a managed gateway node may perform cloud-facing PQC operations for devices with limited resources, subject to the trust model in Section III-B. Third, primitive selection is workload-specific: the architecture considers ML-KEM for session-key establishment, ML-DSA for recurring authentication, and SLH-DSA for selected infrequent operations that benefit from a hash-based signature construction. To minimize communication overhead and latency on resource-constrained edge nodes, this architecture assumes the cloud's public keys are provisioned out-of-band (e.g., during factory flashing or secure onboarding). While this static-key approach trades off Perfect Forward Secrecy (PFS), it strictly bounds the energy and time required for routine session establishment.

E. Cryptographic Workflows: KEM/DEM and Signatures

The architecture distinguishes key-establishment flows from authentication exchanges. ML-KEM encapsulates a shared secret and outputs a KEM ciphertext; a key-derivation step can then supply keying material to an authenticated-encryption-with-associated-data (AEAD) construction that protects the application payload. ML-DSA and SLH-DSA instead produce signatures associated with a message, challenge, command, or firmware artifact. Figure 4 shows both workflows. The reported core benchmarks cover key generation, encapsulation, decapsulation, signing, and verification. DEM processing time is not measured. Communication overhead is therefore modeled analytically as

$$B_{\text{total}} = |c_{\text{KEM}}| + |N| + |C| + |T| \quad (1)$$

where $|c_{\text{KEM}}|$ is the KEM ciphertext size, $|C|$ is the encrypted payload size (equal to the plaintext length), and the accounting example uses a 12-byte nonce ($|N| = 12$) and a 16-byte authentication tag ($|T| = 16$). These values represent one AEAD parameterization, not a measured TLS implementation or a complete network packet. Figure 5 reports the resulting object sizes for three application-payload lengths.

F. Resource-Constrained Compatibility and Native-PQC Deployment Policy

Resource-constrained edge nodes are not assumed to be categorically incapable of public-key operations; optimized PQC implementations may execute on some restricted platforms. In this architecture, however, nodes with tight SRAM, flash, energy, or bandwidth budgets remain in the resource-constrained edge-node class until workload-specific measurements justify admission to the high-capability edge-node class. Their default path therefore uses local authenticated encryption followed by gateway-assisted PQC.

In contrast, high-capability edge nodes and gateway nodes serve as the default candidates for native PQC execution. These entities execute ML-KEM and ML-DSA directly, provided that empirical benchmarking demonstrates acceptable latency, memory, energy, and communication costs. This architectural distinction improves compatibility with legacy and resource-constrained devices while enabling high-capability edge nodes

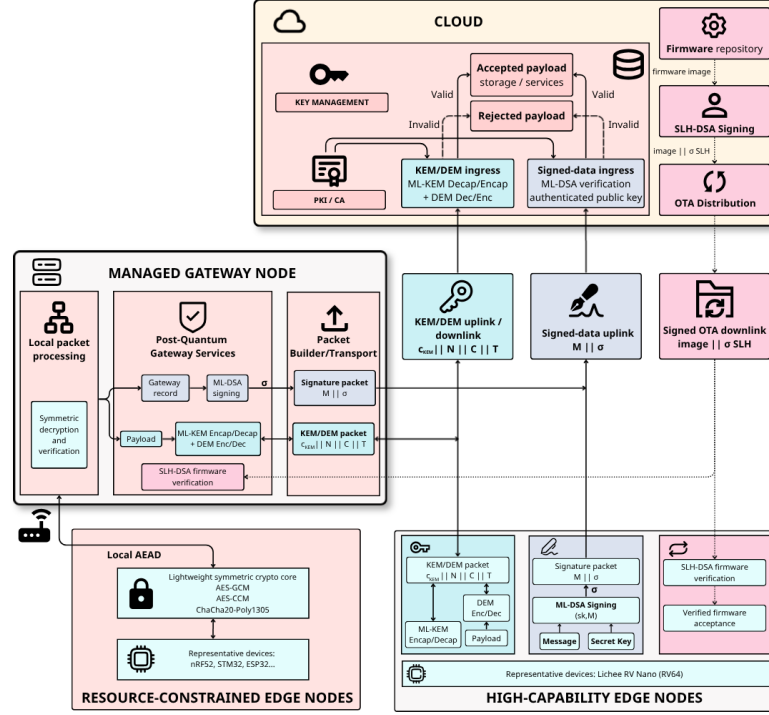


Fig. 3. Capability-tiered cryptographic flows. Resource-constrained edge nodes first protect records with local AEAD and rely on a managed gateway node for verification and policy enforcement. Gateway-origin data or data from high-capability edge nodes then use complete KEM/DEM or signed-data objects for cloud-facing uplinks, while SLH-DSA-signed firmware flows from the cloud to gateway node or high-capability edge node verification before acceptance. Note: Device models and alternative symmetric ciphers are shown for illustrative architectural context; the empirical evaluation focuses on Lichee RV Nano W and AES-GCM.

to communicate directly with cloud services. The resulting deployment policy is formalized in Table II.

G. Gateway-Assisted PQC for Resource-Constrained Edge Nodes

For resource-constrained edge nodes, the proposed workflow avoids frequent native PQC by default. A node protects sensor telemetry or control data within the local domain using an AEAD mechanism supported by the target platform. The gateway node verifies and decrypts the local message, applies access-control and packet-filtering policy, and establishes a cloud-facing channel using ML-KEM-derived keying material. The gateway node may sign an aggregated or forwarded record using ML-DSA, but that signature authenticates gateway-node processing rather than proving that the resource-constrained edge node signed the record.

This design moves cloud-facing public-key computation and associated key management from resource-constrained edge nodes to gateway nodes. It centralizes policy enforcement, update coordination, and security logging, but it does not eliminate scaling: gateway-node capacity must still be provisioned for the number of devices, message rate, and cryptographic workload.

H. Native PQC for High-Capability Edge Nodes

High-capability edge nodes, typically driven by 64-bit processors, may possess the resource capacity to establish

post-quantum-protected communication sessions directly with cloud services. Within this architectural workflow, the edge node utilizes ML-KEM to derive a shared session secret, subsequently employing symmetric encryption to protect application data payloads. When origin authentication or data integrity is required, the node executes ML-DSA directly to sign transmitted messages or verify digital signatures received from cloud services.

This native path removes the gateway node from the end-to-end cryptographic trust path for high-capability edge nodes. It may be appropriate for industrial controllers, local inference nodes, or embedded platforms running a full operating system, provided that platform measurements support the workload. Under the reported implementation, the selected SLH-DSA parameter sets have larger signatures and higher signing cost than ML-DSA; the architecture therefore considers SLH-DSA for selected operations such as firmware validation, secure boot, or root-of-trust establishment rather than frequent telemetry.

I. Cryptographic Primitive-to-Device Mapping

The architecture maps primitives to device roles through conditional deployment rules. It considers ML-KEM for session establishment by gateway nodes, high-capability edge nodes, and cloud services; ML-DSA for recurring authentication on these native-PQC roles; and SLH-DSA selectively at the high-capability edge, gateway, or cloud tier for infrequent firmware-validation, secure-boot, and root-of-trust operations.

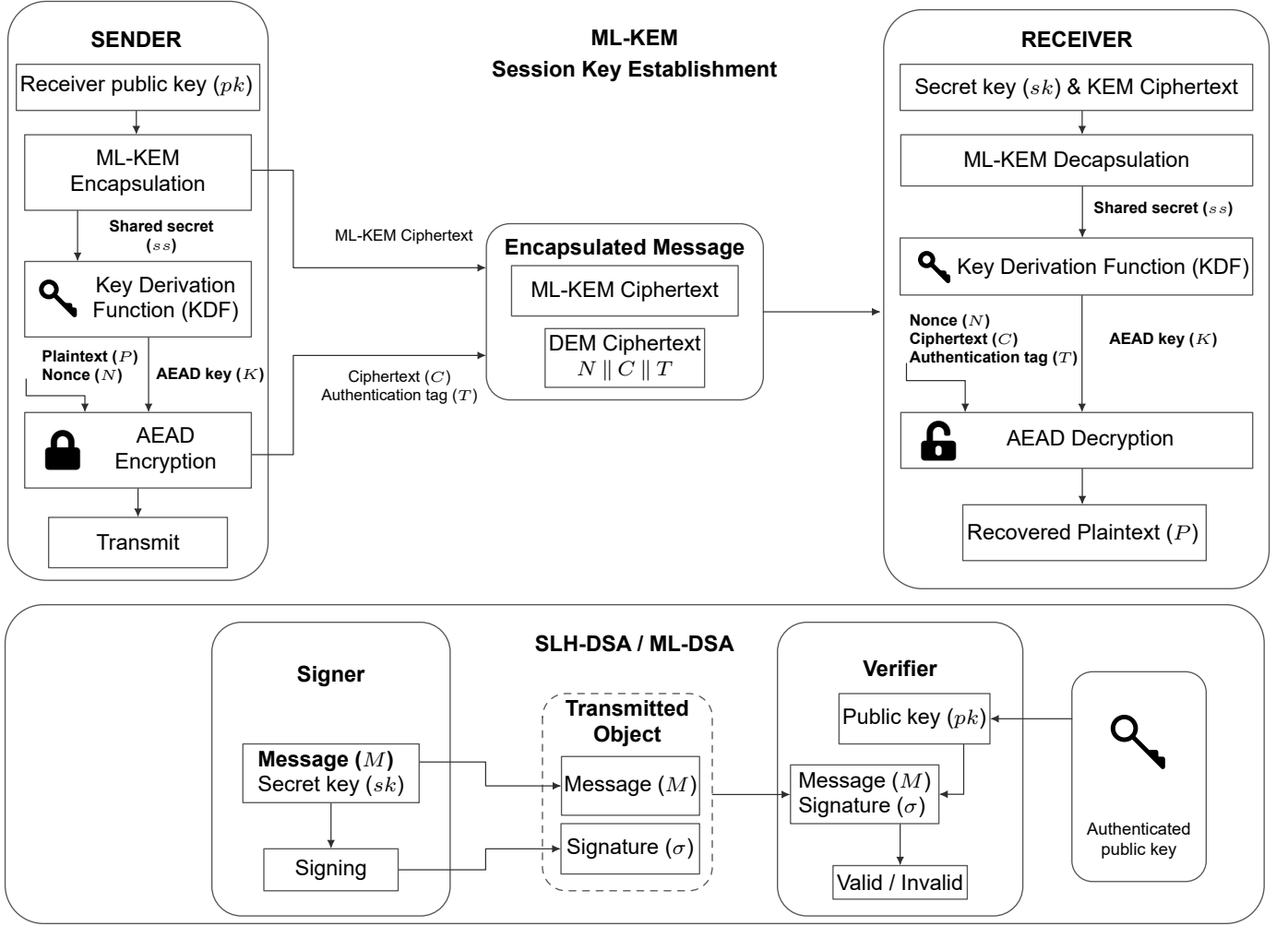


Fig. 4. Cryptographic objects and key flow in the ML-KEM/DEM confidentiality model and the ML-DSA/SLH-DSA signature model. The AEAD object-size parameters are analytical, and DEM processing time is outside the benchmarked cryptographic core.

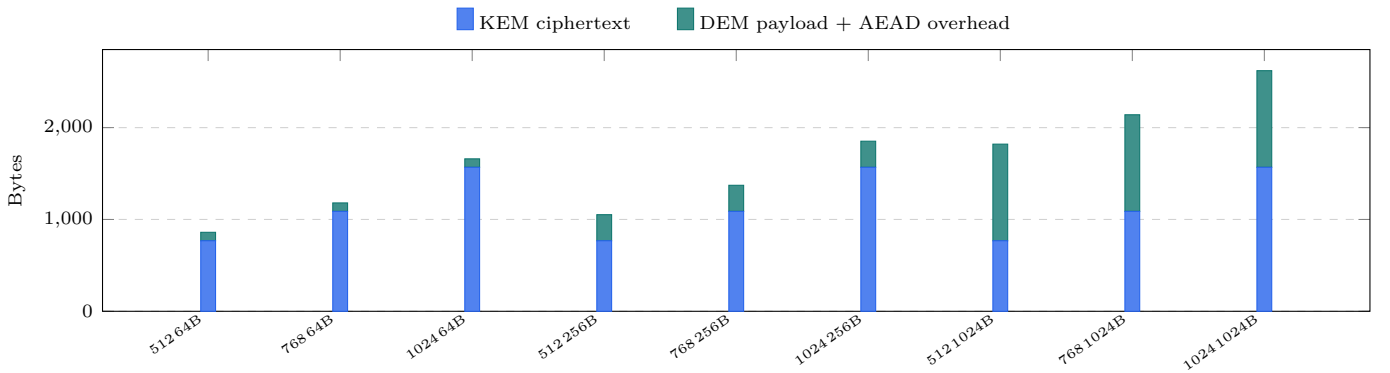


Fig. 5. Analytical ML-KEM/DEM object sizes. Labels combine the ML-KEM parameter set (512, 768, or 1024) and application payload (64, 256, or 1024 B); AEAD overhead is 28 B.

Native edge-node use remains conditional on platform-specific profiling. Table III summarizes these recommendations.

IV. EXPERIMENTAL METHODOLOGY

A. Evaluated Primitives

The evaluated lattice-based configurations are ML-KEM-512, ML-KEM-768, ML-KEM-1024, ML-DSA-44 (implemented using the Dilithium2 parameter set), ML-DSA-65 (Dilithium3),

TABLE II
EXECUTION POLICY BY ARCHITECTURAL ROLE

Architectural Role	Typical Platform	Default PQC Placement	Direct Cloud Access	Rationale
Resource-constrained edge node	Low-memory embedded platform, often 32-bit	Gateway-assisted PQC by default	Not by default	Preserves compatibility and avoids overloading CPU, memory, energy, and low-bandwidth links
High-capability edge node	Workload-validated embedded platform, typically ARM64 or RV64	Native ML-KEM, ML-DSA, and selective SLH-DSA after profiling	Conditional	Enabled only when latency, memory, energy, and communication costs are acceptable
Gateway node	64-bit x86_64, ARM64, or RV64	PQC termination and policy enforcement	Supported	Stable power, larger memory, and better network connectivity
Cloud service	High-resource server or cloud platform	Full PQC endpoint; all primitives by role	N/A (Endpoint)	Sufficient CPU, memory, and network resources to support ML-KEM, ML-DSA, and SLH-DSA

TABLE III
PROPOSED PQC DEPLOYMENT MATRIX FOR IoT SYSTEMS

Architectural Role	Resource Profile	Recommended Cryptographic Path	Recommended PQC Role
Resource-constrained edge node	Low CPU, small SRAM, battery powered, low bandwidth	Local symmetric protection; PQC delegated to a 64-bit gateway node	Avoid direct SLH-DSA; consider ML-KEM/ML-DSA only after platform validation
High-capability edge node	Workload-validated resources, typically a 64-bit CPU and lightweight OS	Native ML-KEM/ML-DSA for routine use; selective SLH-DSA	Direct cloud sessions and authentication; consider SLH-DSA only for infrequent firmware, secure-boot, or Root-of-Trust operations
Gateway node	64-bit CPU, stable power, good network connectivity	PQC gateway service layer between local IoT domain and cloud	ML-KEM for key establishment, ML-DSA for authentication, SLH-DSA for firmware and Root-of-Trust
Cloud service	High resource budget, centralized storage and analytics	Full PQC endpoint	Support ML-KEM, ML-DSA, and SLH-DSA as required

and ML-DSA-87 (Dilithium5). These six configurations form the frequent-operation core for session establishment and routine authentication.

For SLH-DSA, this manuscript uses the six small-signature “s” parameter-set configurations listed in Table IV: SHA2-128s, SHA2-192s, SHA2-256s, SHAKE-128s, SHAKE-192s, and SHAKE-256s. They are included in the physical-node stress records as the hash-based signature family. Because the reported SLH-DSA signing workloads are heavier than the ML-KEM and ML-DSA workloads, they use a lower concurrency setting and are interpreted as evidence for infrequent firmware, secure-boot, or root-of-trust operations rather than high-rate telemetry.

B. Implementation and Compilation Settings

The reported experiments use C implementations from research forks of the Kyber, Dilithium, and SPHINCS+ code bases, mapped to the corresponding ML-KEM, ML-DSA, and SLH-DSA families. This workflow is related to open benchmarking ecosystems such as liboqs and PQCclean [36], [37]. The code is used for performance characterization rather

TABLE IV
SLH-DSA SMALL-SIGNATURE “s” PARAMETER SETS USED IN THE PHYSICAL-NODE STRESS TEST

Variant	Hash family	Level	Sig. (B)
sha2-128s-simple	SHA2	L1	7856
sha2-192s-simple	SHA2	L3	16224
sha2-256s-simple	SHA2	L5	29792
shake-128s-simple	SHAKE	L1	7856
shake-192s-simple	SHAKE	L3	16224
shake-256s-simple	SHAKE	L5	29792

than FIPS validation: the fork Known Answer Test (KAT) files are not claimed to be bit-identical validation artifacts for FIPS 203–205. The physical Lichee RV Nano W records report GNU Make 4.3 and GCC/G++ 11.4.0 on a custom Ubuntu 22.04 LTS Jammy image with Linux 5.10.4 for RV64. ARM64 and RV64 QEMU 8.2.2 runs are limited to functional portability and relative trend checks.

The compilation configuration for the benchmarked primitives is managed via optimized Makefiles. In the research

forks, the compilation toolchains are unified to use gcc rather than generic paths (e.g., /usr/bin/cc) to ensure consistent code-generation behavior across physical and emulated platforms. Key adaptations introduced in the research forks include:

- **SPHINCS+ / SLH-DSA:** The reported research-fork Makefile uses operating-system-specific randomness sources for functional builds. The benchmark targets instead use a deterministic pseudo-random source (rng.c). This choice removes live entropy acquisition from the timed path and makes the benchmark unsuitable as evidence of production randomness handling or end-to-end operational latency.
- **Kyber / ML-KEM:** The reported Makefile adds client-server and multi-session stress targets. The security-level selector uses MODE=2,3,4 for Kyber-512, Kyber-768, and Kyber-1024, respectively.
- **Dilithium / ML-DSA:** The reported Makefile excludes long-running vector-verification targets from the default build while retaining speed, multiplication, and shared-library targets.

The core compilation settings are summarized in Table V.

Table VI consolidates the implementation provenance, build environment, physical-node configuration, workload, power-measurement scope, and emulation limits used to interpret the results.

Exact paper-v1.0 tags and commits pin the benchmark and stress-test source for all three research forks. The releases contain the client-side stress logs for the evaluated algorithms, and the repository provides a verification script that reproduces the corrected aggregates. Consequently, source provenance is pinned for all three families, and primary-record reproducibility is complete for the released session data.

C. Platforms and Measurement Scope

The x86_64 measurements represent native execution on 64-bit platforms that can act as gateway nodes or server-class endpoints. The ARM64 and RV64 QEMU 8.2.2 environments provide functional portability and relative software-trend checks only; they are neither cycle-accurate hardware models nor evidence of absolute board latency. The physical Lichee RV Nano W provides native RV64 Linux evidence for a high-capability edge node under batched concurrent sessions. Cloud services are treated as assumed high-resource endpoints for PQC termination, verification, and management rather than as a separately benchmarked platform. This scope separates native gateway-class evidence, emulated cross-ISA feasibility, physical edge-node behavior, and the architectural cloud role without implying that results transfer unchanged across processors, operating systems, frequency policies, or thermal conditions.

These two evaluation modes are deliberately separate and serve distinct measurement objectives. The core speed benchmarks isolate algorithm-level latency: each primitive is exercised in a tight single-process loop of 10,000 repetitions, with no network stack, no concurrency, and no inter-process coordination, so that the reported mean reflects pure cryptographic operation cost with scheduling variance reduced by repetition count. The physical node-level stress test, by contrast,

evaluates system-level behavior under concurrent load: for ML-KEM and ML-DSA, 80 client sessions run in parallel per batch across 10 batches; for SLH-DSA, 10 sessions run in parallel per batch across 10 batches. The elapsed time in the stress test therefore captures wall-clock duration including process orchestration, OS scheduling, and message-flow handling, and must not be compared directly with the per-operation latency from the speed benchmarks. Values in the stress-test table and figures are reported as arithmetic means across the 10 completed batches. The metrics are elapsed wall-clock time, CPU user time, CPU system time, maximum resident set size (RSS), transmitted-object size, and the whole-board USB input power envelope.

D. Full-Load High-Capability Edge-Node Stress-Test Scenario

In addition to the isolated algorithm speed benchmarks described above, the evaluation reports batched concurrent message flows on an edge-node/server setup. Whereas the speed benchmarks measure single-process, single-operation latency over 10,000 repetitions, the stress test measures aggregate system behavior when the node handles multiple simultaneous cryptographic sessions. For ML-KEM and ML-DSA, each batch contains 80 concurrent sessions; for SLH-DSA, each batch contains 10 concurrent sessions. Each configuration uses 10 batches. The lower concurrency for SLH-DSA is intentional: the heavier signing cost of the selected small-signature parameter sets would make 80 concurrent sessions impractical on the tested node, and the reduced concurrency reflects their expected deployment pattern as infrequent operations rather than high-rate telemetry. The unequal concurrency levels therefore prevent a normalized cross-family performance ranking. Reported timing and memory fields are arithmetic means over the 10 batches, while power is summarized only as a board-level idle-to-active USB input envelope. For signatures, the transmitted cryptographic object is the signature. For ML-KEM, the communication analysis combines the KEM ciphertext with an analytically calculated AEAD payload. The aggregate records are attributed to a Lichee RV Nano W RV64 Linux setup using the environment in Table VI. The tagged releases contain the successful client-side records for the respective configurations.

The stress-test scenario is intended to answer system-level questions: which primitives remain suitable under high message concurrency, which primitives are more appropriately deployed on gateway nodes or server platforms, and at what point the node becomes limited by latency, CPU time, memory, or bandwidth. The RAM value in the stress-test tables is interpreted as Maximum Resident Set Size (RSS) observed for the process, not as embedded peak stack usage or microcontroller SRAM footprint. This distinction prevents overinterpreting Linux process-memory measurements as direct embedded-device memory requirements.

E. Stress-Test Metrics and Interpretation Rules

The stress-test measurements are interpreted at the system level. Table VII defines how every reported field is used in the subsequent result table and figures. The elapsed time captures

TABLE V
REPORTED COMPILATION SETTINGS

Primitive	Compiler	Optimization and Warning Flags
SPHINCS+ / SLH-DSA	gcc	-O3 -Wall -Wextra -Wpedantic -Wconversion -Wmissing-prototypes -Wredundant-decls -Wshadow -Wpointer-arith
Kyber / ML-KEM	gcc	-O3 -fomit-frame-pointer -Wall -Wextra -Wpedantic -Wmissing-prototypes -Wredundant-decls -Wshadow -Wpointer-arith
Dilithium / ML-DSA	gcc	-O3 -fomit-frame-pointer -Wall -Wextra -Wpedantic -Wmissing-prototypes -Wredundant-decls -Wshadow -Wpointer-arith -Wvla

TABLE VI
REPORTED BENCHMARK ENVIRONMENT, ARTIFACT PROVENANCE, AND SCOPE

Field	Recorded Value / Source	Scope Note
PQC implementation	Pinned paper-v1.0 releases of the Kyber (ML-KEM), Dilithium (ML-DSA), and SPHINCS+ (SLH-DSA) research forks, respectively at commits 4d0d05f, 934698d, and e83606b [38], [39], [40]	The tagged trees contain the benchmark and stress-test source, as well as the generated client-side result logs for the evaluation. These research forks are not claimed as Cryptographic Module Validation Program (CMVP)/FIPS validation artifacts or as bit-identical final-standard KATs
Build tools	GNU Make 4.3; GCC/G++ 11.4.0 on the physical RV64 node	Compiler flags are listed in Table V; no platform-specific vector acceleration is claimed for the reported physical-node stress records
Physical node OS	Lichee RV Nano W running a custom Ubuntu 22.04 LTS Jammy image; Linux 5.10.4 #1 Sat Jul 5 08:49:26 UTC 2025 RV64; image/documentation from the Z841973620 LicheeRV-Nano Ubuntu release and RuyiSDK support matrix	The board is treated as the physical RV64 high-capability edge-node target for ML-KEM, ML-DSA, and SLH-DSA stress records
CPU configuration	XuanTie C906-class RV64 execution environment; RV64GCV-capable core family; Linux-visible RV64 userspace; fixed 1.0 GHz CPU-frequency configuration	The tested board image exposes a fixed-frequency operating point rather than a configurable Linux cpufreq governor; therefore, no dynamic governor or turbo-boost state is claimed
Timing and workload	ML-KEM and ML-DSA core loops use 10,000 repetitions; physical stress runs use 80 concurrent sessions $\times$ 10 batches for ML-KEM/ML-DSA and 10 concurrent sessions $\times$ 10 batches for SLH-DSA	Reported stress values are arithmetic means over the 10 completed batches at the selected concurrency; elapsed time remains wall-clock behavior including scheduling/orchestration effects
Power measurement	Whole-board USB input power measured with a KOWSI KWS-2302C USB meter; all tested algorithms and parameter sets stayed within an approximately 0.5 W idle to 0.6 W active envelope, with reading uncertainty on the order of ± 0.1 W	The values represent coarse whole-board USB input power only; they are not per-operation energy measurements and should be used only to show that no large board-level power spike was observed
QEMU configuration	QEMU 8.2.2 for ARM64 and RV64 cross-ISA trend runs	QEMU measurements are used only for functional portability and relative trends, not cycle-accurate physical-board timing

the wall-clock behavior of the stress run and may include scheduling, buffering, process orchestration, and message-flow handling. It should therefore not be read as a pure cryptographic primitive latency. CPU user time and CPU system time provide a narrower view of computation and kernel interaction, respectively. Maximum RSS reports the largest resident memory observed for the Linux process during the run. It is useful for comparing process-level memory trends, but it is not equivalent to MCU SRAM consumption.

The transmitted-object columns are also interpreted by primitive family. For ML-DSA, the signature column is the dominant digital-signature object attached to the tested challenge or message. For ML-KEM, the KEM ciphertext column reports the PQC encapsulation object, while the DEM

payload is calculated separately using Eq. (1). This separation prevents the benchmark from mixing PQC-core timing with symmetric encryption timing, while still allowing the paper to reason about realistic transmitted bytes.

V. RESULTS AND DISCUSSION

A. Communication Metadata Overhead

Communication overhead matters for IoT systems with low-bandwidth links and limited packet buffers. This analysis compares ML-KEM ciphertexts, ML-DSA signatures, and SLH-DSA small-signature “s” parameter-set sizes. For ML-KEM, Eq. (1) and Figure 5 define the analytical KEM/DEM accounting instead of repeating the same calculation in a separate table. The ciphertext sizes are 768, 1,088, and

TABLE VII
INTERPRETATION OF STRESS-TEST METRICS

Metric	Interpretation in this paper
Elapsed time	Full-run wall-clock time, including orchestration and concurrency effects
CPU user	User-mode CPU time consumed by the process
CPU sys	Kernel/system CPU time associated with the process
RAM	Linux process Maximum RSS; not MCU SRAM or stack usage
KEM ct	ML-KEM ciphertext object
Signature	ML-DSA or SLH-DSA signature object
DEM payload	Analytical AEAD payload; excluded from PQC-core timing
Power	Whole-board USB input power envelope measured at the USB supply input; approximately 0.5 W idle and 0.6 W active across the tested parameter sets

1,568 B, and the selected AEAD parameterization adds a fixed 28 B nonce-and-tag overhead to the application payload. The resulting transmitted totals are 860–1,660 B for a 64 B payload and 1,820–2,620 B for a 1,024 B payload. Table VIII lists the cryptographic-object sizes across security categories.

Table VIII shows that the listed ML-KEM ciphertexts are below 2 KB and the ML-DSA signatures are below 5 KB, whereas the selected SLH-DSA small-signature parameter sets range from 7,856 to 29,792 B. Object size is only one deployment factor; latency, implementation security, message rate, and link-layer overhead must also be considered.

B. SLH-DSA Physical-Node Scope

The physical-node stress table contains six small-signature “s” parameter sets: SHA2-128s, SHA2-192s, SHA2-256s, SHAKE-128s, SHAKE-192s, and SHAKE-256s. It does not mix these records with fast “f” variants, which have different latency and signature-size trade-offs.

The reported workload does not support high-rate use of the selected SLH-DSA small-signature parameter sets on the tested node. SLH-DSA elapsed means range from approximately 136 to 518 s for 10 concurrent sessions, compared with approximately 1.0 to 2.0 s for the 80-session ML-KEM and ML-DSA batches. Because concurrency and operations differ, this is a workload-level placement observation rather than a normalized per-operation comparison. The QEMU 8.2.2-based emulation environment remains useful for checking functional portability and observing relative execution trends across ARM64 and RV64 models. It is not used in this manuscript to derive or claim absolute physical-board latency. The physical Lichee RV Nano W records constitute the primary node-level evidence, while the QEMU 8.2.2-based emulation environment supports the cross-ISA feasibility discussion.

C. Full-Load High-Capability Edge-Node Stress-Test Results

The node-level stress results report mean elapsed time, CPU time, maximum RSS, and transmitted-object sizes. The evaluation methodology strictly measures the cryptographic core (Encapsulation/Decapsulation and DEM) under the assumption that long-term public keys are pre-provisioned on the edge nodes, thereby eliminating the RTT overhead of public-key distribution during session establishment. Table IX

TABLE VIII
COMMUNICATION OBJECT SIZES

Primitive	Level	Object	Bytes
ML-KEM-512	L1	KEM ciphertext	768
ML-KEM-768	L3	KEM ciphertext	1088
ML-KEM-1024	L5	KEM ciphertext	1568
ML-DSA-44	L2	Signature	2420
ML-DSA-65	L3	Signature	3309
ML-DSA-87	L5	Signature	4627
SLH-DSA-128s	L1	Signature	7856
SLH-DSA-192s	L3	Signature	16224
SLH-DSA-256s	L5	Signature	29792

gives the aggregate records. ML-KEM and ML-DSA use 80 concurrent sessions per batch, whereas SLH-DSA uses 10; each configuration uses 10 batches. The settings are therefore interpreted as unequal workload records, not a normalized per-operation ranking. The pinned artifacts contain the successful per-session records for each evaluated parameter set [38], [39], [40]. Recalculation by ciphertext length reproduces the reported CPU, RSS, and object-size columns and identifies an elapsed-time transposition in an earlier manuscript version: the corrected means are 2039.45 ms for ML-KEM-768 and 1926.38 ms for ML-KEM-1024.

Because every ML-KEM batch contains 80 records, the mean of the 800 session records equals the mean of the ten consecutive batch means. Treating each consecutive 80-record block in the released file as one batch gives elapsed-time standard deviations of 170.08, 1030.96, and 140.34 ms for Kyber512, Kyber768, and Kyber1024, respectively. The Kyber768 dispersion is driven by one batch whose mean is 4965.74 ms; the other nine batch means range from 1584.68 to 1873.95 ms. This post hoc grouping is reproducible from file order but should be strengthened by explicit batch identifiers in future logs.

The reported per-session records show three bounded observations. First, the selected ML-KEM and ML-DSA workloads complete in approximately 1.0–2.0 s, whereas the unequal-concurrency SLH-DSA workloads require approximately 136–518 s. It is also notable that the 192-bit security SLH-DSA variants execute slower than their 256-bit

TABLE IX
REPORTED PHYSICAL-NODE PER-SESSION STRESS-TEST MEANS

Primitive	Sess.	Batch	Elapsed (ms)	Client CPU usr (ms)	Client CPU sys (ms)	Client RSS (KiB)
ML-DSA-44	80	10	1047.11	7.22	0.90	564.00
ML-DSA-65	80	10	1419.34	11.50	1.13	596.00
ML-DSA-87	80	10	1446.34	14.00	1.36	576.00
ML-KEM-512	80	10	1783.63	1.83	0.74	768.00
ML-KEM-768	80	10	2039.45	2.29	0.77	772.00
ML-KEM-1024	80	10	1926.38	2.87	0.98	844.00
SLH-DSA-SHA2-128s	10	10	135925.17	11416.68	9.92	608.00
SLH-DSA-SHA2-192s	10	10	308673.02	26123.06	31.03	637.48
SLH-DSA-SHA2-256s	10	10	296101.86	24835.22	32.71	568.00
SLH-DSA-SHAKE-128s	10	10	296865.17	24937.65	21.49	512.00
SLH-DSA-SHAKE-192s	10	10	518074.23	43346.48	38.70	584.00
SLH-DSA-SHAKE-256s	10	10	450439.57	37764.38	34.10	660.00

counterparts (e.g., SHA2-192s at roughly 308.6 s versus SHA2-256s at 296.1 s). This inversion is a known phenomenon in hash-based signatures stemming from hardware alignment and optimization differences between 24-byte (192-bit) and 32-byte (256-bit) internal hash states on specific processor architectures. Second, maximum RSS lies between approximately 512 and 844 KiB; because this is Linux process RSS and the repository contains neither allocation traces nor a board memory-limit analysis, it does not identify the hardware bottleneck. Third, USB input power is reported near 0.5 W at idle and 0.6 W during active runs. This measurement reflects the system-level power envelope rather than the dynamic micro-joule cost of individual cryptographic operations. The roughly 0.1 W difference is comparable to the stated meter uncertainty and cannot support an algorithm-level energy ranking, but it does confirm that continuous PQC workloads do not cause prohibitive power spikes or brownouts on the tested node.

Figure 6 visualizes the mean per-session client-side elapsed and user-CPU metrics from Table IX; logarithmic axes are necessary because the selected SLH-DSA workloads are orders of magnitude slower than the ML-KEM and ML-DSA workloads.

Figure 7 visualizes per-client process CPU time and maximum RSS.

For ML-KEM, the stress table reports the KEM ciphertext and shared-secret size; the DEM payload is intentionally calculated through the analytical model in Eq. (1). For ML-DSA and SLH-DSA, the signature dominates the transmitted cryptographic object. These measured and analytical results motivate the placement rules in Table X: ML-KEM and ML-DSA form the frequent-operation layer, while SLH-DSA is considered for firmware validation, secure boot, update authorization, and root-of-trust establishment paths.

D. Deployment Implications

Table VIII quantifies the communication requirement behind the tiered placement policy. ML-KEM ciphertexts occupy 768–1,568 B, ML-DSA signatures occupy 2,420–4,627 B, and the selected SLH-DSA signatures occupy 7,856–29,792 B

before protocol and link-layer overhead. Table IX and Figure 6 link those sizes to the measured workload: each ML-KEM and ML-DSA configuration represents 10 batches of 80 sessions (800 sessions total) and reports a mean per-session elapsed time of approximately 1.0–2.0 s, whereas each SLH-DSA configuration represents 10 batches of 10 sessions (100 sessions total) and requires approximately 136–518 s per session. Because concurrency and cryptographic operations differ, these values support workload-specific placement but not a normalized cross-family ranking.

Within this evidence boundary, ML-KEM is the stronger candidate for recurring shared-secret establishment and ML-DSA for recurring authentication on gateway-class platforms and high-capability edge nodes admitted after profiling. The selected SLH-DSA small-signature variants are better reserved for infrequent firmware validation, secure boot, update authorization, or root-of-trust operations. This is a conditional deployment implication from the tested implementations and workloads, not proof that the same ranking holds for every platform or protocol.

E. Architecture Support Against Stress-Test Objectives

The stress-test scenario evaluates batched node-to-server work rather than a single isolated primitive call. Table IX and Figure 7 show that all tested configurations report a mean per-client maximum RSS of approximately 512–844 KiB and a coarse whole-board USB input envelope of about 0.5 W idle and 0.6 W active. The roughly 0.1 W difference is comparable to the meter uncertainty and therefore cannot establish an algorithm-level energy ranking. The pinned paper-v1.0 artifacts supply the stress-test source and corresponding session records for the evaluation.

Table X converts these bounded observations into deployment rules, but it does not demonstrate growth or capacity under increasing offered load. Gateway queueing delay, sustainable arrival rate, saturation point, post-saturation recovery, and aggregate throughput across multiple boards were not measured. The gateway-assisted policy therefore reduces cloud-facing public-key work on constrained devices, but no claim is made

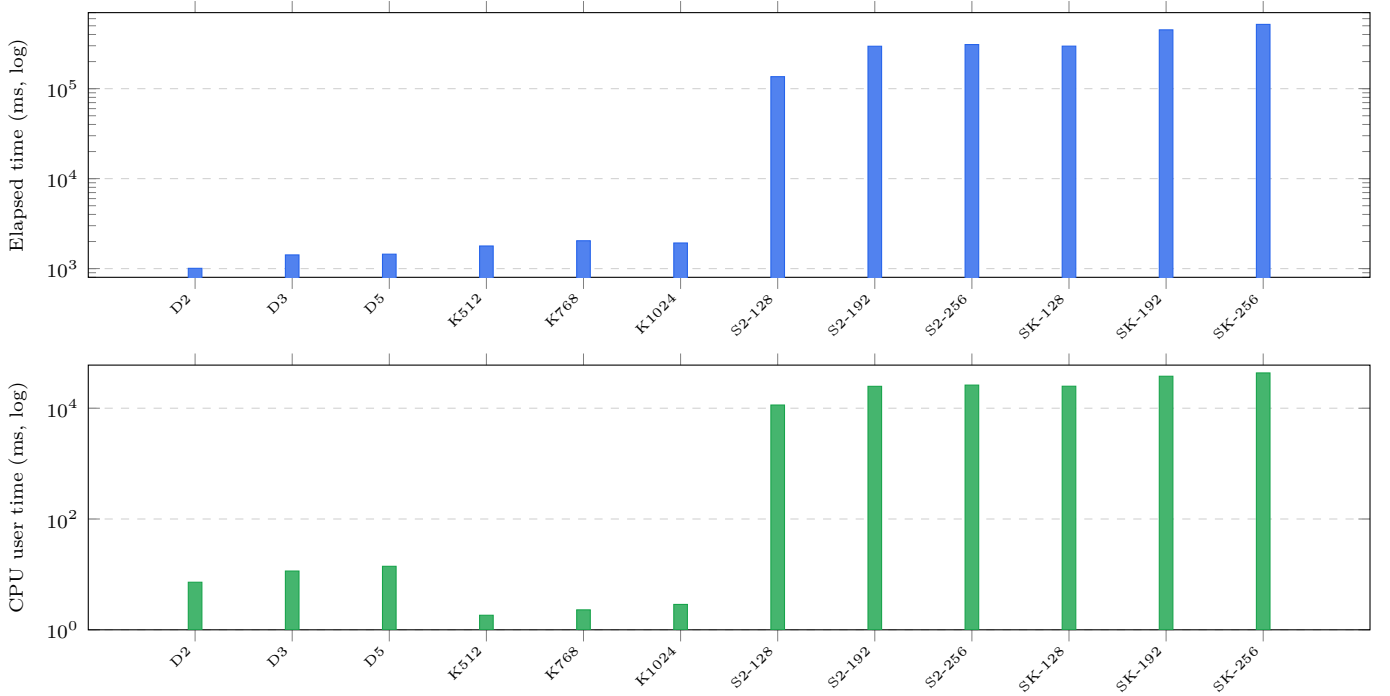


Fig. 6. Reported mean per-session client-side metrics for elapsed and user CPU time. D2/D3/D5 denote Dilithium2/3/5; K denotes Kyber; S2 and SK denote SHA2- and SHAKE-based small-signature SPHINCS+ variants. Axes are logarithmic; concurrency differs by family.

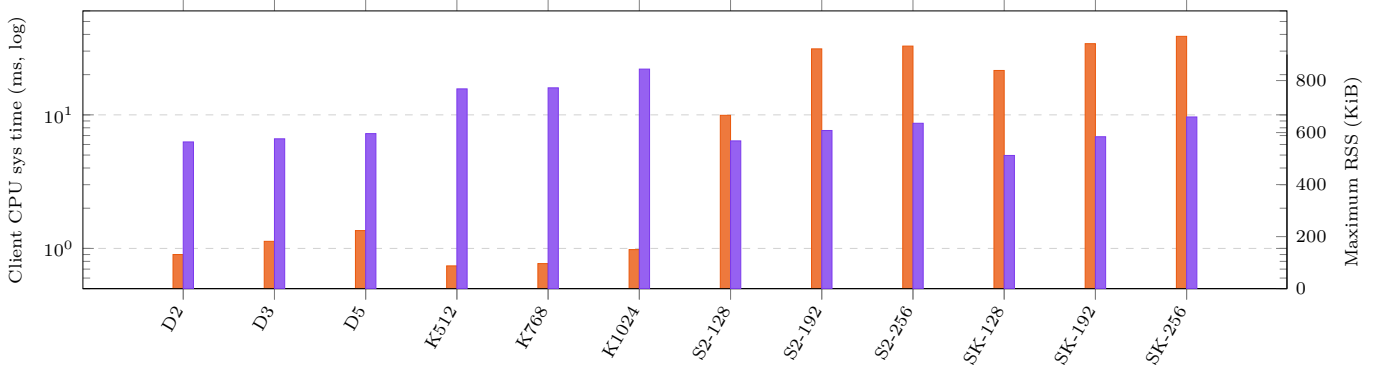


Fig. 7. Reported per-client process CPU sys time (left logarithmic axis) and maximum resident set size (right linear axis) for the mean per-session metrics. Abbreviations follow Fig. 6.

TABLE X
DEPLOYMENT RULES FROM THE EVALUATION

Rule	Recommended Decision	Reason
Frequent session establishment	Use ML-KEM at gateway nodes, high-capability edge nodes, and cloud services	Low ciphertext size and low latency make it suitable for recurring channel setup
Routine message authentication	Use ML-DSA at gateway nodes and high-capability edge nodes admitted after platform-specific profiling	Signature size and latency are more suitable than SLH-DSA for regular traffic
Resource-constrained edge-node operation	Use local symmetric protection and delegate PQC to a 64-bit gateway node	Preserves compatibility and avoids overloading CPU, memory, and wireless bandwidth
Firmware and Root-of-Trust operations	Use SLH-DSA selectively at gateway nodes, high-capability edge nodes, or cloud services	Hash-based signatures provide conservative assurance but are expensive for frequent traffic
Native-PQC eligibility for edge nodes	Assign the high-capability edge-node path only after device-specific profiling	Feasibility depends on memory, energy, implementation, compiler, and workload

that it removes system bottlenecks as the number of devices or message rate grows.

The stress-test data should therefore be interpreted as feasibility and placement evidence rather than a capacity benchmark or pure cryptographic ranking. A primitive acceptable for occasional firmware validation may remain unsuitable for telemetry, and a primitive feasible on a server may still be too expensive for a resource-constrained edge node. Future evaluation must sweep arrival rate and concurrency, report queue length and tail latency, identify saturation and recovery behavior, and measure multi-board throughput before making quantitative scalability claims.

VI. CONCLUSION AND FUTURE WORK

A. Limitations and Future Directions

This work has five empirical limitations. First, QEMU 8.2.2 is used only for functional portability and relative trend checks; it is not a cycle-accurate substitute for physical ARM64 or RV64 hardware. Native physical-node evidence is limited to one Lichee RV Nano W RV64 configuration. Second, maximum resident set size is a Linux process metric, not peak stack, MCU SRAM, or bare-metal memory consumption. Third, SLH-DSA uses 10 concurrent sessions per batch, whereas ML-KEM and ML-DSA use 80, preventing normalized cross-family ranking. Fourth, the power record is a coarse whole-board USB input envelope rather than per-operation energy. Fifth, exact tagged benchmark source is available for all three families, but only the released ML-KEM session log permits direct recalculation; the tagged ML-DSA and SLH-DSA trees contain stress-test source without generated result logs. ML-KEM dispersion can be reconstructed post hoc from file order, while explicit batch identifiers and a definition of the x86_64 aggregate-latency operation set remain unavailable as structured artifacts. The architecture is designed to adapt by admitting high-capability edge nodes after profiling and provisioning gateway-node capacity; throughput and queueing performance have not yet been measured.

Future work will extend this stress-testing methodology to multi-board distributed deployments, incorporate native physical ARM64 measurements, provide supplementary per-batch logs with explicit batch identifiers, and integrate highly optimized implementations using ARM NEON, the RISC-V Vector Extension, and dedicated hardware cryptographic accelerators. Further directions include PQC-TLS 1.3 integration for real-world IoT protocols (MQTT, CoAP), side-channel resistance evaluation, and a systematic sweep of arrival rate and concurrency to characterize saturation and throughput scaling.

B. Conclusion

This paper presented a capability-tiered architecture for deploying post-quantum cryptography in resource-heterogeneous IoT environments. Resource-constrained edge nodes use authenticated local-domain protection and a managed gateway node for cloud-facing PQC, while high-capability edge nodes may execute ML-KEM and ML-DSA natively after platform-specific profiling. By delegating cloud-facing PQC to the gateway, the architecture isolates edge nodes from post-quantum overheads,

intentionally trading direct device-to-cloud cryptographic origin for local-domain hardware compatibility.

Under the reported implementations and workloads, ML-KEM and ML-DSA complete 80-session concurrent batches in approximately 1.0–2.0 s per session with transmitted objects below 5 KB, while the selected SLH-DSA small-signature variants require approximately 136–518 s for 10-session batches with signatures of 7,856–29,792 B. This supports considering ML-KEM and ML-DSA for recurring session establishment and authentication, and SLH-DSA for selected infrequent operations such as firmware validation and root-of-trust establishment. The result is a conditional placement guideline, not a universal ranking; deployment still requires device-specific profiling, protocol-level evaluation, and reproducible measurements.

REFERENCES

- [1] D. Sikeridis, P. Kampanakis, and M. Devetsikiotis, "Post-quantum authentication in TLS 1.3: A performance study," in *Proceedings of the 2020 Network and Distributed System Security Symposium*, Internet Society, 2020. doi: 10.14722/ndss.2020.24203 [Online]. Available: <https://eprint.iacr.org/2020/071>
- [2] A. Holgado-Moreno, J. Portilla, D. López-Fernández, and L. Redondo, "Energy consumption benchmarking of post-quantum cryptography on resource-constrained IoT nodes," *IEEE Internet of Things Journal*, vol. 13, no. 9, pp. 19 562–19 573, 2026. doi: 10.1109/JIOT.2026.3663839
- [3] Z. Liu, K.-K. R. Choo, and J. Grossschadl, "Securing edge devices in the post-quantum internet of things using lattice-based cryptography," *IEEE Communications Magazine*, vol. 56, no. 2, pp. 158–162, 2018. doi: 10.1109/MCOM.2018.1700330
- [4] A. Khalid, S. McCarthy, W. Liu, and M. O'Neill, *Lattice-based cryptography for IoT in a quantum world: Are we ready?* Cryptology ePrint Archive, Paper 2019/681, 2019. [Online]. Available: <https://eprint.iacr.org/2019/681>
- [5] C. A. Roma, C.-E. A. Tai, and M. A. Hasan, "Energy efficiency analysis of post-quantum cryptographic algorithms," *IEEE Access*, vol. 9, pp. 71 295–71 317, 2021. doi: 10.1109/ACCESS.2021.3077843
- [6] National Institute of Standards and Technology, "Module-lattice-based key-encapsulation mechanism standard," National Institute of Standards and Technology, Gaithersburg, MD, USA, Federal Information Processing Standards Publication FIPS 203, Aug. 2024. doi: 10.6028/NIST.FIPS.203 [Online]. Available: <https://doi.org/10.6028/NIST.FIPS.203>
- [7] National Institute of Standards and Technology, "Module-lattice-based digital signature standard," National Institute of Standards and Technology, Gaithersburg, MD, USA, Federal Information Processing Standards Publication FIPS 204, Aug. 2024. doi: 10.6028/NIST.FIPS.204 [Online]. Available: <https://doi.org/10.6028/NIST.FIPS.204>

- [8] National Institute of Standards and Technology, "Stateless hash-based digital signature standard," National Institute of Standards and Technology, Gaithersburg, MD, USA, Federal Information Processing Standards Publication FIPS 205, Aug. 2024. doi: 10.6028/NIST.FIPS.205 [Online]. Available: <https://doi.org/10.6028/NIST.FIPS.205>
- [9] B. Halak, T. Gibson, M. Henley, C.-B. Botea, B. Heath, and S. Khan, "Evaluation of performance, energy, and computation costs of quantum-attack resilient encryption algorithms for embedded devices," *IEEE Access*, vol. 12, pp. 8791–8805, 2024. doi: 10.1109/ACCESS.2024.3350775
- [10] A. Magyari and Y. Chen, "Optimizing SPHINCS+ for low-power devices," *Electronics*, vol. 14, no. 17, 2025, Art. no. 3460. doi: 10.3390/electronics14173460 [Online]. Available: <https://www.mdpi.com/2079-9292/14/17/3460>
- [11] M. Abbasi, F. Cardoso, P. Váz, J. Silva, and P. Martins, "A practical performance benchmark of post-quantum cryptography across heterogeneous computing environments," *Cryptography*, vol. 9, no. 2, 2025, Art. no. 32. doi: 10.3390/cryptography9020032 [Online]. Available: <https://www.mdpi.com/2410-387X/9/2/32>
- [12] G. Fitzgibbon and C. Ottaviani, "Constrained device performance benchmarking with the implementation of post-quantum cryptography," *Cryptography*, vol. 8, no. 2, 2024, Art. no. 21. doi: 10.3390/cryptography8020021 [Online]. Available: <https://www.mdpi.com/2410-387X/8/2/21>
- [13] J. Bos, J. Renes, and A. Sprenkels, "Dilithium for memory constrained devices," in *Progress in Cryptology – AFRICACRYPT 2022*, ser. Lecture Notes in Computer Science, vol. 13503, Springer, 2022, pp. 217–235. doi: 10.1007/978-3-031-17433-9_10
- [14] D. O. C. Greconici, M. J. Kannwischer, and A. Sprenkels, "Compact dilithium implementations on Cortex-M3 and Cortex-M4," *IACR Transactions on Cryptographic Hardware and Embedded Systems*, vol. 2021, no. 1, pp. 1–24, 2020. doi: 10.46586/tches.v2021.i1.1-24 [Online]. Available: <https://tches.iacr.org/index.php/TCHES/article/view/8725>
- [15] M. J. Kannwischer, J. Rijneveld, P. Schwabe, and K. Stoffelen, *pqm4: Testing and benchmarking NIST PQC on ARM cortex-m4*, Cryptology ePrint Archive, Paper 2019/844, 2019. [Online]. Available: <https://eprint.iacr.org/2019/844>
- [16] Z. Ye, R. Song, H. Zhang, D. Chen, R. C.-C. Cheung, and K. Huang, "A highly-efficient lattice-based post-quantum cryptography processor for IoT applications," *IACR Transactions on Cryptographic Hardware and Embedded Systems*, vol. 2024, no. 2, pp. 130–153, 2024. doi: 10.46586/tches.v2024.i2.130-153 [Online]. Available: <https://tches.iacr.org/index.php/TCHES/article/view/11423>
- [17] X. Carril et al., "Hardware acceleration for high-volume operations of CRYSTALS-Kyber and CRYSTALS-Dilithium," *ACM Transactions on Reconfigurable Technology and Systems*, vol. 17, no. 3, 2024. doi: 10.1145/3675172 [Online]. Available: <https://doi.org/10.1145/3675172>
- [18] A. Abdulrahman et al., "Towards ML-KEM & ML-DSA on OpenTitan," in *2025 IEEE Symposium on Security and Privacy*, IEEE, 2025, pp. 1–19. doi: 10.1109/SP61157.2025.00220 [Online]. Available: <https://eprint.iacr.org/2024/1192>
- [19] M. A. Ehsan, W. Alayed, A. U. Rehman, W. u. Hassan, and A. Zeeshan, "Post-quantum KEMs for IoT: A study of kyber and NTRU," *Symmetry*, vol. 17, no. 6, 2025, Art. no. 881. doi: 10.3390/sym17060881 [Online]. Available: <https://www.mdpi.com/2073-8994/17/6/881>
- [20] R. Chhetri, *Benchmarking NIST-standardised ML-KEM and ML-DSA on ARM Cortex-M0+: Performance, memory, and energy on the RP2040*, arXiv preprint arXiv:2603.19340, 2026. arXiv: 2603.19340. [Online]. Available: <https://arxiv.org/abs/2603.19340>
- [21] R. Chhetri, *PQC benchmarks on ARM Cortex-M0+ (RP2040)*, 2026. doi: 10.5281/ZENODO.19393777 [Online]. Available: <https://zenodo.org/doi/10.5281/zenodo.19393777>
- [22] J. Bozhko, Y. Hanna, R. Harrilal-Parchment, S. Tonyali, and K. Akkaya, "Performance evaluation of quantum-resistant TLS for consumer IoT devices," in *2023 IEEE 20th Consumer Communications & Networking Conference (CCNC)*, 2023, pp. 230–235. doi: 10.1109/CCNC51644.2023.10060762
- [23] M. Schöffel, F. Lauer, C. C. Rheinländer, and N. Wehn, "On the energy costs of post-quantum KEMs in TLS-based low-power secure IoT," in *Proceedings of the International Conference on Internet-of-Things Design and Implementation*, ser. IoTDI '21, Association for Computing Machinery, 2021, pp. 158–168. doi: 10.1145/3450268.3453528 [Online]. Available: <https://doi.org/10.1145/3450268.3453528>
- [24] M. Schöffel, F. Lauer, C. C. Rheinländer, and N. Wehn, "Secure IoT in the era of quantum computers—where are the bottlenecks?" *Sensors*, vol. 22, no. 7, 2022, Art. no. 2484. doi: 10.3390/s22072484 [Online]. Available: <https://www.mdpi.com/1424-8220/22/7/2484>
- [25] G. Tasopoulos, J. Li, A. P. Fournaris, R. K. Zhao, A. Sakzad, and R. Steinfeld, *Performance evaluation of post-quantum TLS 1.3 on resource-constrained embedded systems*, Cryptology ePrint Archive, Paper 2021/1553, 2021. [Online]. Available: <https://eprint.iacr.org/2021/1553>
- [26] M.-J. O. Saarinen, "Mobile energy requirements of the upcoming NIST post-quantum cryptography standards," in *2020 8th IEEE International Conference on Mobile Cloud Computing, Services, and Engineering (MobileCloud)*, 2020, pp. 23–30. doi: 10.1109/MobileCloud48802.2020.00012
- [27] G. Banegas, K. Zandberg, A. Herrmann, E. Baccelli, and B. Smith, *Quantum-resistant security for software updates on low-power networked embedded devices*, 2021. arXiv: 2106.05577 [cs.CR]. [Online]. Available: <https://arxiv.org/abs/2106.05577>

- [28] M. Nielsen, M. Kjeldsen, T. Turnip, and B. Andersen, "Post-quantum digital signature algorithms on IoT: Evaluating performance on LoRa ESP32 microcontroller," in *Proceedings of the 22nd International Conference on Security and Cryptography*, SCITEPRESS, 2025, pp. 592–600. doi: 10.5220/0013508400003979
- [29] M. Vidaković and K. Milićević, "Performance and applicability of post-quantum digital signature algorithms in resource-constrained environments," *Algorithms*, vol. 16, no. 11, 2023, Art. no. 518. doi: 10.3390/a16110518 [Online]. Available: <https://www.mdpi.com/1999-4893/16/11/518>
- [30] U. Banerjee, T. S. Ukyab, and A. P. Chandrakasan, "Sapphire: A configurable crypto-processor for post-quantum lattice-based protocols," *IACR Transactions on Cryptographic Hardware and Embedded Systems*, vol. 2019, no. 4, pp. 17–61, 2019. doi: 10.13154/tches.v2019.i4.17-61 [Online]. Available: <https://tches.iacr.org/index.php/TCHES/article/view/8344>
- [31] V. B. Dang, F. Farahmand, M. Andrzejczak, K. Mohajerani, D. T. Nguyen, and K. Gaj, *Implementation and benchmarking of round 2 candidates in the NIST post-quantum cryptography standardization process using hardware and software/hardware co-design approaches*, Cryptology ePrint Archive, Paper 2020/795, 2020. [Online]. Available: <https://eprint.iacr.org/2020/795>
- [32] P. Karl, J. Schupp, and G. Sigl, "Performance and communication cost of hardware accelerators for hashing in post-quantum cryptography," *ACM Transactions on Embedded Computing Systems*, vol. 24, no. 5, 2025. doi: 10.1145/3676965 [Online]. Available: <https://doi.org/10.1145/3676965>
- [33] G. Mao et al., "High-performance and configurable SW/HW co-design of post-quantum signature CRYSTALS-Dilithium," *ACM Transactions on Reconfigurable Technology and Systems*, vol. 16, no. 3, 2023. doi: 10.1145/3569456 [Online]. Available: <https://doi.org/10.1145/3569456>
- [34] P. Karl, J. Schupp, T. Fritzmann, and G. Sigl, "Post-quantum signatures on RISC-V with hardware acceleration," *ACM Transactions on Embedded Computing Systems*, vol. 23, no. 2, pp. 1–23, 2024. doi: 10.1145/3579092 [Online]. Available: <https://eprint.iacr.org/2022/538>
- [35] S. Zhang, Y. Zhao, X. Yu, and J. Han, "RISC-V domain-specific processor for accelerating SPHINCS+ on multi-core architecture," in *2024 IEEE 17th International Conference on Solid-State & Integrated Circuit Technology (ICSICT)*, 2024, pp. 1–3. doi: 10.1109/ICSICT62049.2024.10830991
- [36] Open Quantum Safe Project. "liboqs: Open-source c library for quantum-safe cryptographic algorithms," Accessed: Jul. 14, 2026. [Online]. Available: <https://github.com/open-quantum-safe/liboqs>
- [37] M. J. Kannwischer, P. Schwabe, D. Stebila, and T. Wiggers, "Improving software quality in cryptography standardization projects," in *Proceedings of the IEEE European Symposium on Security and Privacy Workshops*, ser. EuroS&PW, Genoa, Italy: IEEE, 2022, pp. 19–30. doi: 10.1109/EuroSPW55150.2022.00010 [Online]. Available: <https://doi.org/10.1109/EuroSPW55150.2022.00010>
- [38] Q. Nguyen. "Kyber research-fork benchmark artifact, release paper-v1.0." Tagged commit 4d0d05f, Accessed: Jul. 14, 2026. [Online]. Available: <https://github.com/quannnguyen247/kyber-dev/releases/tag/paper-v1.0>
- [39] Q. Nguyen. "Dilithium research-fork benchmark artifact, release paper-v1.0." Tagged commit 934698d, Accessed: Jul. 14, 2026. [Online]. Available: <https://github.com/quannnguyen247/dilithium-dev/releases/tag/paper-v1.0>
- [40] Q. Nguyen. "SPHINCS+ research-fork benchmark artifact, release paper-v1.0." Tagged commit e83606b, Accessed: Jul. 14, 2026. [Online]. Available: <https://github.com/quannnguyen247/sphincsplus-dev/releases/tag/paper-v1.0>